

Lab 3 Practice: LEDs Controller

This practice does not need to be submitted.

Objective

- 1 This exercise will help you gain hands-on experience with FPGA LEDs, clock division, and finite-state machine (FSM) design in SystemVerilog.

Practice Tasks

1. lab3_practice.sv

In this lab, you must design an LED controller based on a finite-state machine (FSM). Then you can control the LED sequence using switches, as described below.

a. I/O list

I/O	Connected to	Definition
clk	W5	Clock signal with the frequency of 100MHz
rst	SW15	Asynchronous active-high reset signal, resetting FSM to INITIAL state
slow	SW0	Enables the slow LED flashing
fast	SW1	Enables the fast LED flashing
end_light	SW2	Ends the light sequence and transitions FSM to FINAL state
led[15:0]	LD15~LD0	Lighting up the LEDs in the sequence based on the control of the switches

b. FSM Specification

The FSM should consist of four states: **INITIAL**, **LOAD**, **PLAY**, and **FINAL**.

- At any time, **rst == 1** asynchronously resets the FSM to the **INITIAL** state.
- When **rst==0**, the FSM resumes normal operation.

✓ INITIAL state

- Entered when **rst == 1**.
- All LEDs (**LD15~LD0**) are **on**.

LD15 ●●●●●●●●●●●●●● LD0

(● : LED on, ○ : LED off)

- During normal operation, the FSM transitions from the **INITIAL** state to the **LOAD** state at the next rising edge of **clk**.

✓ LOAD state

- Implement a 4-second loading sequence. Starting with all LEDs off, light one additional LED every 0.25 seconds, from LD0 toward LD15.
- Each step lights one additional LED.

LD15 ○○○○○○○○○○○○○○○○● LD0
 LD15 ○○○○○○○○○○○○○○○○●● LD0
 LD15 ○○○○○○○○○○○○○○○○●●● LD0
 ...
 LD15 ●●●●●●●●●●●●●●●● LD0
 (● : LED on, ○ : LED off)

- After all 16 LEDs are ON, the FSM transitions to the **PLAY** state at the next rising edge of **clk**.

✓ **PLAY** state

- If **end_light** == 1, the FSM transitions to the **FINAL** state at the next rising edge of **clk**.
- If **end_light** == 0, the FSM remains in the **PLAY** state.
- In this state, the LED behavior depends on switch settings:
 - **Slow Mode (slow==1):**
Even-numbered LEDs are controlled by a divided clock with a frequency of $100\text{MHz}/2^{28}$.

LD15 ○●○●○●○●○●○●○●○● LD0
 (● : LED on, ○ : LED off)
 LD15 ○○○○○○○○○○○○○○○○ LD0
 (● : LED on, ○ : LED off)

- **Fast Mode (fast==1):**
Odd-numbered LEDs are controlled by a divided clock with a frequency of $100\text{MHz}/2^{27}$.

LD15 ●○●○●○●○●○●○●○●○ LD0
 (● : LED on, ○ : LED off)
 LD15 ○○○○○○○○○○○○○○○○ LD0
 (● : LED on, ○ : LED off)

- If **slow** and **fast** are both set to 1:
 - Even LEDs follow slow flashing.
 - Odd LEDs follow fast flashing.

✓ **FINAL** state

- All 16 LEDs flash together, toggling every 0.5 seconds.

- After three full flashing cycles (i.e., on → off → on → off → on → off; 3 seconds total), the FSM returns to the **INITIAL** state.

LD15 ●●●●●●●●●●●●●● LD0 (0.5 seconds ON)

(● : LED on, ○ : LED off)

LD15 ○○○○○○○○○○○○○○ LD0 (0.5 seconds OFF)

(● : LED on, ○ : LED off)

You must use the following template for your design:

```
module lab3_practice (
    input logic clk,
    input logic rst,
    input logic slow,
    input logic fast,
    input logic end_light,
    output logic[15:0] led
);
    // add your design here
endmodule
```

Hints

1. Develop the habit of **drawing a block diagram before writing code**. For this practice lab, we provide a reference block diagram; however, you are strongly encouraged to design the hardware architecture yourself first. **A block diagram will not be provided in the basic lab.**
2. Before implementing the FSM block, it is also suggested to **draw the FSM diagram** first. This can help you clearly define the states, transition conditions, and output behavior before writing code.
3. In class, we learned how to generate a slower clock using a **clock divider**. However, a power-of-two clock divider cannot generate an **exact one-second interval** from a 100 MHz clock. For precise time intervals, such as 0.25 seconds, you may need to implement a **timer**. A recommended approach is to generate a one-clock-cycle pulse at a fixed interval.

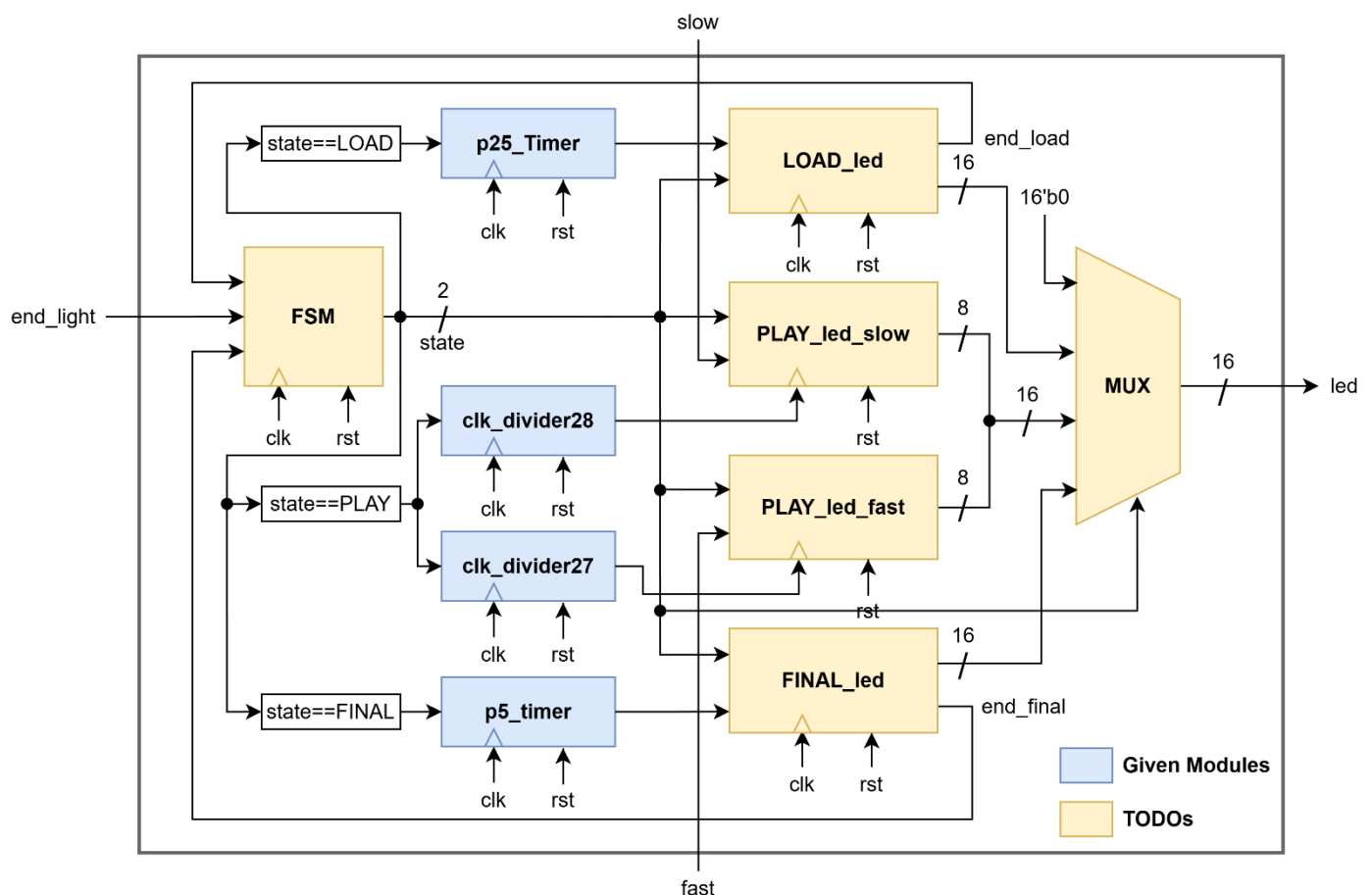
- For example, **0.25 seconds** corresponds to **25,000,000 clock cycles** when the input clock frequency is 100 MHz. You can implement a **sequential counter** that counts clock cycles and asserts a pulse for **exactly one clock cycle** when it reaches the target count.
- The pulse can then be used as an **enable signal** to update the FSM or LED state.

The template provided for this lab includes implementations of both a clock divider and a timer. **Make sure you understand how they work and how to implement an FSM**, as you will use them very frequently in future labs.

Attention

- ✓ This practice prepares you for the next lab. Make sure you fully understand your design.
- ✓ Feel free to ask questions about the specification on the EECLASS forum.
- ✓ DO NOT copy-and-paste code segments from PDFs (hidden characters may cause hard-to-debug syntax errors).

Reference Block Diagram



Try replacing the clock divider enable condition `state == PLAY` with `state == PLAY && slow` (or `state == PLAY && fast`) and observe how the behavior changes.

TA's Reminder

Make sure you practice this lab thoroughly. Basic lab 3 will be **more challenging** than Lab 2. So make sure you understand the FSM, clock divider, and timer designs.